1. General structure

1. What is the name of the test that is run by default when you run the command "make all"?

The default test that is run when we execute the make all command is specified in the TEST variable, which is defined as pss_spi_interrupt_test. Therefore, the test run by default is **pss_spi_interrupt_test**.

By default, the make file runs a pss_spi_interrupt_test, but there are others available - see the tests defined in the test directory.

2. What other tests can you run?

By default, the make file runs a pss_spi_interrupt_test, but there are others available. To run a different test, you would set the TEST variable either via the command line or modify it in the Makefile to the desired test case.

make run - Run the simulation in command line mode - we can also run the following tests by adding **make all TEST=<test name>**

Name	Last commit	Last update
..		
pss_gpio_outputs_test.svh	Integration level task files	1 week ago
pss_gpio_outputs_test.svh.bck	Integration level task files	1 week ago
pss_spi_interrupt_test.svh	Integration level task files	1 week ago
pss_spi_interrupt_test.svh.bck	Integration level task files	1 week ago
pss_spi_polling_test.svh	Integration level task files	1 week ago
pss_test.svh	Integration level task files	1 week ago
pss_test_base.svh	Integration level task files	1 week ago
pss_test_lib_pkg.sv	Integration level task files	1 week ago

In our case we can run the following:

- pss_gpio_outputs_test
- pss_spi_polling_test
- pss_test
- pss_test_base
- pss_test_lib_pkg

3. Describe the DUT.

The Design Under Test (DUT) in this integration-level testbench consists of a Peripheral Sub-System (PSS) that integrates two block-level verification environments: SPI and GPIO.

Key Components of the DUT:

- SPI block: A Serial Peripheral Interface (SPI) communication protocol environment.
- GPIO block: A General Purpose Input/Output (GPIO) environment.

- AHB to APB Bus Bridge: An intermediary that connects the AHB to the APB interfaces. This bus bridge is part of the integration to handle the communication between the SPI and GPIO blocks.
- PSS: The PSS contains the SPI and GPIO blocks, connected via the AHB to APB bus bridge. The APB bus interfaces are no longer directly exposed but are monitored by passive APB agents within the testbench.

4. Which parts of the DUT are verified by this testbench? Which are not?

Verified by the Testbench:

- SPI Block: The SPI peripheral's functionality is verified by this testbench. This includes communication over the SPI interface and ensuring the correct handling of transactions.
- GPIO Block: The GPIO block is also verified. This includes ensuring proper operation for GPIO inputs and outputs and monitoring the interface during operation.
- AHB to APB Bus Bridge: The interface between the AHB and APB buses is tested. The testbench ensures that the AHB interface correctly communicates with the APB interfaces of the SPI and GPIO blocks.
- PSS Integration: The integration of SPI and GPIO into the PSS and the operation of the PSS with the bus bridge is verified. This includes ensuring the SPI and GPIO blocks work together correctly through the AHB to APB bus bridge.

Not Verified by the Testbench:

- Non-exposed APB Interfaces: The testbench does not verify the APB bus interfaces directly as these are put in passive mode by the APB agents, meaning no active stimulus is driven onto the APB interface during this test.

5. Describe the topology of the UVM environment.

We got the topology print in terminal by adding the print command in pss_test file under the run phaser and we could see the topology prints by running the simulation using the command line make all TEST= pss_test.

The UVM topology task print_topology displays all instantiated components in the environment and helps in debug and to identify if any component got left out.

From our output log of the UVM testbench topology, detailing the components in the testbench environment. The log shows the hierarchy and relationships between various UVM components in the environment, such as agents, monitors, sequencers, predictors, and scoreboards.

Below you can see the break down of the topology:

1. Root of the Topology: uvm_test_top (pss_test)
2. Environment Level: m_env (pss_env)
3. First Sub-Environment:
    - m_ahb2reg_predictor (uvm_reg_predictor #(ahb_seq_item))
    - m_ahb_agent (ahb_agent):

        - m_driver (ahb_driver):
        - m_monitor (ahb_monitor):
        - m_sequencer (ahb_sequencer):
        - Queues:
    4. Second Sub-Environment:
        - m_gpio_env (gpio_env):
            - m_GPI_agent (gpio_agent)
            - m_GPOE_agent (gpio_agent) and m_GPO_agent (gpio_agent)
            - m_apb2reg_predictor (uvm_reg_predictor #(apb_seq_item))
    5. Additional Sub-Components:
        - m_in_sb (gpio_in_scoreboard)
        - m_out_sb (gpio_out_scoreboard)
    6. SPI Environment:
        - m_spi_env (spi_env)
            - m_apb2reg_predictor (uvm_reg_predictor #(apb_seq_item))
            - m_apb_agent (apb_agent)
            - m_scoreboard (spi_scoreboard)
            - m_spi_agent (spi_agent)
                - m_driver (spi_driver)
                - m_monitor (spi_monitor)
                - m_sequencer (uvm_sequencer)

2. Connectivity

1. What are the top level modules given for the simulator? How are they connected?

The top-level modules for the simulator are:

- **hdl_top**: This module instantiates the DUT, the BFM interfaces such as APB, SPI, GPIO, and connects the pin interfaces to the DUT and BFM interfaces. It also manages the clock and reset signals for the design and handles the binding of internal signals for monitoring.
- **hvl_top**: This module runs the UVM test by importing the test library and invoking the run_test() function.

Connections:

- The hdl_top module connects various interfaces like APB, AHB, SPI, GPIO to the DUT.
- BFM interfaces are instantiated to interact with these interfaces.
- The virtual interfaces for the monitors and drivers are configured and set using the uvm_config_db.
- The hvl_top runs the tests, initiating the simulation and managing the test phases.

2. Describe the DUT interfaces.

The DUT interfaces are the connections through which the DUT communicates with the testbench or other components. Here's a simple breakdown of the main interfaces:

- **APB (Advanced Peripheral Bus)**: Used for low-speed communication with peripherals. It includes signals like address (PADDR), data (PWDATA, PRDATA), and control signals (PWRITE, PSEL).
- **SPI (Serial Peripheral Interface)**: A high-speed interface for serial data transfer. Key signals are clock (SCK), data input (MISO), data output (MOSI), and slave select (SS).
- **GPIO (General Purpose Input/Output)**: Used for simple input or output signals to control devices like LEDs or sensors.
- **AHB (Advanced High-performance Bus)**: A high-speed bus for communication with memory or peripherals. It includes address (HADDR), data (HWDATA, HRDATA), and control signals (HWRITE, HSEL).

These interfaces are connected to the DUT through BFM that manage data transfer and control operations.

3. What is the purpose of Bus Functional Models (BFM)? How are they implemented in this testbench?

A BFM is a high-level simulation model used to simulate the behavior of a bus or interface. The purpose of BFMs in the testbench is to:

- Drive Stimulus: BFMs generate the signals needed for the testbench to interact with the DUT.
- Monitor Interface Activity: BFMs also act as monitors, observing the traffic on the bus and ensuring that the transactions are occurring correctly.
- Abstract Interface: BFMs provide an abstraction layer between the actual interface of the DUT and the testbench, making it easier to simulate and verify functionality without having to directly interact with the hardware at the gate level.

How BFMs are implemented in this testbench:
In the testbench, BFMs are implemented as model components that generate and respond to bus transactions. They simulate communication protocols and handle signal timing, data transfer, and control signal management.
For example:

- APB BFM would simulate the read/write operations on the APB bus and manage the address, data, and control signals like PWRITE and PSEL.
- SPI BFM would simulate the serial data communication and handle the timing of MISO, MOSI, and SS signals.
- GPIO BFM is used to drive values to the GPIO pins and monitor the behavior of the GPIO block during simulation.

3. Configuration

1. How is configuration done for the testbench?

Configuration is done using configuration objects that hold settings for components. These configuration objects are stored in the uvm_config_db. Components retrieve settings using uvm_config_db::get().

2. What kind of configuration objects are there?

- pss_env_config: Configuration for the entire environment.
- spi_env_config: Configuration for SPI-related components.
- gpio_env_config: Configuration for GPIO-related components.
- ahb_agent_config: Configuration for AHB agent.
- apb_agent_config: Configuration for APB agent.

3. Which components are affected by configuration?

- Environments: (e.g., pss_env, spi_env, gpio_env).
- Agents: (e.g., spi_agent, gpio_agent, ahb_agent, apb_agent).
- BFMs: (e.g., SPI BFM).

4. Agents

1. What kinds of agents are available?

- Driver Agents: Responsible for driving stimulus to the DUT.
- Monitor Agents: Responsible for observing the DUT and collecting data.
- Sequencer Agents: Manages the flow of sequences to drivers.
- Interface Agents: Used to represent specific interfaces (e.g., AXI, SPI, UART).

2. Which agents are used in the PSS environment?

In the PSS environment, the following agents are typically used:

- APB Agent: For APB interface interactions.
- AHB Agent: For AHB interface interactions.
- SPI Agent: For SPI interface communication.
- GPIO Agent: For GPIO interface interaction.

3. Which agents are set as active, which as passive? Are there differences between tests?

Active Agents:

- Driver Agents: These agents actively generate and drive stimulus to the DUT. They are responsible for initiating transactions.
- Sequencer Agents: They actively manage and control the flow of sequences to the driver agents. While they don't drive signals directly, they are considered active because they control the stimulus generation.

Passive Agents:

- Monitor Agents: These agents passively observe the signals from the DUT, collecting and reporting data. They don't generate any stimulus, so they are passive.

- Interface Agents: These can sometimes be passive if they are only observing interface traffic and don't interact with the DUT.

Differences Between Tests:
In a write test, the driver agents are typically set as active because they need to drive stimulus to the DUT. In a read or verification test, monitor agents are typically set as active since the goal is to observe and verify DUT behavior. Depending on the test scenario, agents may be toggled between active and passive states to fulfill specific requirements.

5. Constraints

1. What kind of constraints can you find? What is their purpose?

The constraints are used to control and guide the behavior of the testbench, ensuring that the design is tested correctly and in a structured manner. The main types of constraints include:

- Interface Constraints: These define how different parts of the system communicate with each other. For example, the testbench includes various interfaces like apb_if, ahb_if, and spi_if, which help in connecting different blocks (DUT and BFMs). This ensures proper connections between the DUT and the testbench components such as the monitors, drivers, and agents.
- Agent Constraints: These constraints are related to the agents driving the interfaces. For example, the m_spi_apb_agent_cfg and m_gpio_apb_agent_cfg are configured to be in passive mode. They control the behavior of the agents, such as whether they actively generate signals or just observe the system.
- Configuration Constraints: The pss_test_base class configures various sub-components like SPI, GPIO, and AHB agents. This configuration ensures that the components interact correctly during the test.
- Monitor Constraints: The monitors like APB_SPI_mon_bfm and APB_GPIO_mon_bfm are configured to observe the bus signals and collect data. They capture and monitor the activity on the interfaces to check if the design is behaving as expected.
- Functional Coverage Constraints: These measure the completeness of the test coverage, ensuring that all relevant aspects of the DUT are exercised during the tests.

6. Covergroups

1. What kind of functional coverage information is collected? Where?

- Assertion Coverage: Checks if assertions were triggered in the design. Found in the Assertion Coverage section of the report.
- Directive Coverage: Checks if specific conditions, often related to assertions or directives, were triggered. Found in the Directive Coverage section.
- Covergroup Coverage: Monitors whether specific combinations of design states (bins) were covered during simulation. Found in the Covergroup Coverage section.

All of this functional coverage information is collected and reported by instance in the design, and it's important for evaluating how effectively the simulation tests have exercised different parts of the design.

2. Modify the Makefile to save a coverage database file and generate a coverage report. Inspect the report after running different tests.

We modified the makefile and got the coverage report using the command make coverage. Total covergroups are 8.

3. What seems to be the reason for these numbers?

Covergroup Coverage: 0.00%
This means that none of the conditions the covergroups are looking for were triggered in the simulation. This can happen if:

- The tests didn’t cover all the necessary scenarios.
- Some parts of the design were never tested.
- The testbenches weren’t set up to trigger those conditions.

Directive Coverage: 50.00%
There are two directives in the report, and one was fully covered while the other wasn't. The directive for /hdl_top/APB was triggered 128,273 times, which means this part of the design was tested properly. The directive for /hdl_top/APB_dummy was never triggered, meaning that part of the design wasn't tested during this simulation. This could be because:

- The tests didn’t include /hdl_top/APB_dummy.
- It wasn’t active during the simulation.

Assertion Coverage: Some assertions pass and others fail.
Assertions are checks to make sure certain conditions are met. Some passed like for /hdl_top/APB, but others didn't like for /hdl_top/APB_dummy.

Total Coverage by Instance: 44.44%
Overall, the design coverage is quite low, meaning many parts of the design weren’t tested. This could be due to:

- The tests didn’t cover all areas of the design.
- Some parts of the design like /hdl_top/APB_dummy were not used or not tested.
- The test scenarios didn’t simulate enough different situations to cover everything.

7. Register Abstraction Layer

1. How is RAL used in this testbench?

The RAL is extensively utilized in our testbench to manage and verify both GPIO and SPI registers systematically.

- Register Modeling:

- Each register (e.g., gpio_in_reg, ctrl_reg, divider_reg) is modeled as a subclass of uvm_reg. These classes define register fields (uvm_reg_field) with attributes such as size, reset value, and access type RO for Read-Only, RW for Read-Write.
- The use of build() methods in these classes ensures that register fields are systematically configured and initialized.

- Register Block:
  - The gpio_reg_block and spi_reg_block classes aggregate their respective registers to represent the hierarchical structure of the DUT's register set.
  - These blocks extend uvm_reg_block to provide unified access to all related registers.
- Register Map:
  - Each register block has a corresponding register map, which assigns specific memory addresses to registers for systematic access during simulation and testing.
  - These maps also include attributes, making them suitable for efficient interaction in diverse environments.
- Coverage Support:
  - The RAL includes coverage mechanisms to track register interactions and ensure thorough testing. Examples include coverage for field values (cg_vals in some classes) to verify proper utilization.

2. Draw a diagram of the register map

gpio_reg_block_map

```
│
├── Address: 'h0   → gpio_in      (RO - Read-Only)
├── Address: 'h4   → gpio_out     (RW - Read-Write)
├── Address: 'h8   → gpio_oe      (RW - Read-Write)
├── Address: 'hc   → gpio_inte    (RW - Read-Write)
├── Address: 'h10  → gpio_ptrig (RW - Read-Write)
├── Address: 'h14  → gpio_aux   (RW - Read-Write)
├── Address: 'h18  → gpio_ctrl    (RW - Read-Write)
├── Address: 'h1c  → gpio_ints    (RW - Read-Write)
├── Address: 'h20  → gpio_eclk   (RW - Read-Write)
└── Address: 'h24  → gpio_nec     (RW - Read-Write)
```

	Standard	Standard	Standard	Standard	Standard	Standard	Standard	Standard	Standard	Standard	Standard	Standard	Standard	Standard
1	Register Name	Register Description	Register Address	Register Width	Register Access	Register Reset Value	Register Constraints	Register Custom Type	Field Name	Field Description	Field Offset	Field Width	Field Access	Field Reset Value
2	gpio_in	GPIO Input	0x0	32	RO	0x0								
3	gpio_out	GPIO Output	0x04	32	RW	0x0								
4	gpio_oe	GPIO Output Enable	0x08	32	RW	0x0								
5	gpio_inte	GPIO Interrupt	0x0C	32	RW	0x0								
6	gpio_ptrig	GPIO PTRIG	0x010	32	RW	0x0								
7	gpio_aux	GPIO Aux	0x014	32	RW	0x0								
8	gpio_ctrl	GPIO Control	0x018	32	RW	0x0			padding	Reserved	2	30	RW	0x0
9	gpio_ctrl								INTS	Interrupt Status	1	1	RW	0x0
10	gpio_ctrl								INTE	Interrupt Enable	0	1	RW	0x0
11	gpio_ints	GPIO Ints	0x01c	32	RW	0x0								
12	gpio_eclk	GPIO eclk	0x020	32	RW	0x0								
13	gpio_nec	GPIO nec	0x024	32	RW	0x0								

spi_reg_block_map

```
│
├── Address: 'h0   → rxtx0        (RW - Read-Write)
```

├── Address: 'h4 → rxtx1 (RW - Read-Write)
├── Address: 'h8 → rxtx2 (RW - Read-Write)
├── Address: 'hc → rxtx3 (RW - Read-Write)
├── Address: 'h10 → ctrl (RW - Read-Write)
├── Address: 'h14 → divider (RW - Read-Write)
└── Address: 'h18 → ss (RW - Read-Write)

	Standard	Standard	Standard	Standard	Standard	Standard	Standard	Standard	Standard	Standard	Standard	Standard	Standard	Standard	Standard
1	Register Name	Register Description	Register Address	Register Width	Register Access	Register Reset Value	Register Constraints	Register Custom Type	Field Name	Field Description	Field Offset	Field Width	Field Access	Field Reset Value	Field is Co
2	rxtx0	RXTX0	0x0	32	RW	0x0									
3	rxtx1	RXTX1	0x04	32	RW	0x0									
4	rxtx2	RXTX2	0x08	32	RW	0x0									
5	rxtx3	RXTX3	0x0C	32	RW	0x0									
6	ctrl	Control Status Register	0x010	32	RW	0x0			ass		13	1	RW		TRUE
7	ctrl								ie		12	1	RW		TRUE
8	ctrl								lsb		11	1	RW		TRUE
9	ctrl								tx_neg		10	1	RW		TRUE
10	ctrl								rx_neg		9	1	RW		TRUE
11	ctrl								go_bsy		8	1	RW		TRUE
12	ctrl								reserved2		7	1	RW		FALSE
13	ctrl								char_len		0	7	RW		TRUE
14	divider	Divider	0x014	32	RW	0x0			ratio		0	16	RW		TRUE
15	ss	Status	0x018	32	RW	0x0			reserved		8	8	RW		FALSE
16	ss								cs		0	8	RW		TRUE

3. Are there backdoor accesses to registers? What does a backdoor access mean?

Yes, backdoor accesses are supported in the testbench for both GPIO and SPI registers. The uvm_reg infrastructure provides direct simulation-level access to register values. This bypasses the frontdoor, hardware interfaces like SPI or GPIO protocols. Register classes are fully configured with uvm_reg_field objects, enabling efficient read/write operations via backdoor paths.

Example:

- For the GPIO package, backdoor methods can directly read/write fields like gpio_in.F.
- For the SPI package, fields such as ctrl.ass or divider.ratio can be accessed directly.

Backdoor access allows testbench components to interact with registers directly through simulation, bypassing hardware communication protocols like SPI or GPIO.

- Faster Debugging: Allows direct read/write operations on registers, accelerating testing and debugging.
- No Protocol Overhead: Unlike frontdoor accesses, backdoor accesses directly modify register values, avoiding delays associated with protocol handling.
- Use in Simulations: Backdoor methods are particularly useful in simulations for tasks like verifying reset values, forcing specific register values, or validating register functionality.

8. Tests

Inspect the pss_interrupt_test:

1. What kind of sequence is run? Draw a diagram of sequence hierarchy.

The sequence being run in the pss_spi_interrupt_test class is an SPI interrupt test. Specifically, it creates an instance of spi_int_vseq, which is a sequence of

transactions related to SPI interrupts. The test runs this sequence 10 times in a repeat loop inside the run_phase task.

```
pss_test_base
    └── pss_test_seq_base
            ├── gpio_outputs_vseq
            └── spi_int_vseq
                    ├── ctrl_go_seq
                    ├── SPI_config_rand_order_seq
                    ├── tfer_over_by_poll_seq
                    ├── spi_rand_seq
                    ├── ahb_write_seq
                    └── ahb_read_seq
```

2. How is a write to AHB bus implemented in the sequence? Explain the full flow from sequence item to DUT pins.

A write to the AHB bus is implemented in the spi_int_vseq::body task with the following code:

```
ahb_write.addr = 32'h200;  // Set the AHB address
ahb_write.data = 32'h1;    // Set the data to be written
ahb_write.start(ahb);      // Start the AHB write transaction
```

The full flow from sequence item to DUT pins can be described as follows:

- Sequence Item Creation: A sequence item like ahb_write_seq, spi_rand_seq, or ctrl_go_seq is created as an object in the test sequence (spi_int_vseq).
- Sequence Execution:
  - When the sequence item (ahb_write) is executed by calling start(ahb), the sequencer (ahb) starts driving the corresponding signals on the DUT's AHB interface.
  - The sequencer controls the flow of data through the virtual interface to the DUT by sequencing different transactions like writes (ahb_write) or reads (ahb_read).
- Interfacing with DUT:
  - For the AHB write operation, the AHB sequencer drives the address and data onto the AHB bus (pins on the DUT). The DUT responds according to the functionality defined for the AHB interface.
  - For SPI, the SPI sequencer drives the SPI-related control signals, data, and clock onto the SPI interface of the DUT.
- Interrupt Handling: The sequence includes handling interrupts by setting up the interrupt controller, checking if the interrupt is triggered, and verifying that the interrupt is cleared via register read/write operations.
- Verification:
  - The test sequence includes checks to ensure the interrupt was properly triggered and cleared by the DUT.
  - It uses m_cfg.wait_for_interrupt to wait for the interrupt and m_cfg.is_interrupt_cleared() to verify if the interrupt is cleared after a read/write operation.

Inspect all the tests:

3. **Do the tests find any kinds of errors in the DUT?**

   pss_gpio_outputs_test: No, the tests passed successfully with no errors detected in the DUT. The output confirms no scoreboard errors, no interrupt read errors, and no output errors during the GPIO tests.

   pss_spi_interrupt_test: Yes, the test run indicates an error. A UVM_FATAL error occurs because the test pss_spi_interrupts_test was not properly registered with the UVM factory, and it could not be created or executed.

   pss_spi_polling_test: No, the tests completed successfully with no errors or fatal issues reported in the DUT. The UVM report confirms that the test passed without any problems.

   pss_test: No, the tests completed successfully without any errors or fatal issues reported in the DUT. The UVM report confirms that the test passed.

   pss_test_base: No, the tests completed successfully, and there were no errors or fatal issues detected in the DUT. The UVM report confirms the test passed without problems.

   pss_test_lib_pkg: Yes, the test run encountered issues. A UVM_FATAL error occurred because the pss_test_lib_pkg component was not registered with the factory, and the test could not be executed.

4. **Describe the purpose of each test: What is tested, what kind of sequences are run?**

   pss_gpio_outputs_test: The purpose of the pss_gpio_outputs_test is to validate GPIO functionality. The test uses a sequence called GPO_test_vseq. The sequence is instantiated and run 10 times in the run_phase task. It runs sequences to check GPIO Output Enable and GPIO Output signals. It ensures no errors are generated during multiple iterations of the test sequences. Virtual interfaces are connected and used during the test for sequence execution.

   pss_spi_interrupt_test: The pss_spi_interrupt_test is designed to validate the SPI interrupt functionality. It runs sequences (spi_int_vseq) to verify proper handling of SPI interrupt signals. Sequencers would be assigned to drive and monitor the relevant interfaces e.g., SPI agent. However, due to the registration error, the intended sequences were not executed in this run.

   pss_spi_polling_test: The pss_spi_polling_test verifies SPI polling functionality. It runs polling sequences (config_polling_test) to test proper configuration and operation of SPI communication. The sequences interact with the SPI environment and SPI agent to ensure the expected behaviors.

pss_test: The pss_test validates register reset values. It runs the uvm_reg_hw_reset_seq sequence to verify the reset values of various registers in the GPIO register block. The sequence ensures that the registers match their expected reset states in the associated memory maps.

pss_test_base: The pss_test_base serves as a base test class to configure and validate the environment for SPI, GPIO, and other interfaces. It sets up configurations for various agents like SPI, GPIO, AHB, etc. and their respective components. The environment includes scoreboards, register models, and interrupt utilities to ensure proper interaction and coverage. This specific test appears to validate functionality in a passive mode, ensuring no errors are reported during basic environment setup.

pss_test_lib_pkg: The pss_test_lib_pkg package is intended to provide a collection of tests for verifying SPI, GPIO, and other functionalities. It includes the following tests:

- GPIO Outputs Test: Validates GPIO output signals using specific sequences.
- SPI Polling Test: Checks SPI communication functionality with polling sequences.
- SPI Interrupt Test: Tests SPI interrupt handling capabilities using interrupt sequences.